0098. Validate Binary Search Tree

dium | Category: Trees | LeetCode: https://leetcode.com/problems/validate-binary

Problem

- Is the tree a valid BST? Every node in the left subtree must be strictly less, and every node in the right subtree strictly greater, recursively.

Intuition

- Comparing a node only with its direct children is not enough; a deep node must respect all its ancestors.
- Pass down an open interval (low, high) that the node's value must lie in.

Brute Force

- For each node, check max(left subtree) < val < min(right subtree): O(n^2).
- Or do an inorder traversal and check it is strictly increasing (O(n), also optimal).

Optimal Approach

- Valid(node, low, high): null → true.
- If val <= low or val >= high → false.
- Return valid(left, low, val) && valid(right, val, high).

Worked Example

```
[5,1,4,null,null,3,6].

5 in (-inf, inf) ok
1 in (-inf, 5) ok
4 in (5, inf)? no -> false
```

Java Solution

```java
public class ValidateBinarySearchTree {
    public boolean isValidBST(TreeNode root) { return valid(root, Long.MIN_VALUE, Long.MAX_VALUE); }

    private boolean valid(TreeNode n, long low, long high) {
        if (n == null) return true;
        if (n.val <= low || n.val >= high) return false;
        return valid(n.left, low, n.val) && valid(n.right, n.val, high);
    }
}
```

Complexity

- Time: O(n). Space: O(h).

Pitfalls

- Values may equal Integer.MIN/MAX_VALUE; use long bounds (or nullable Integer).
- Duplicates are invalid (strict inequality).